

이동형

DevOps Engineer | 이력서 & 경력기술서 · 경력 4년 3개월

E-mail: genius5711@gmail.com Blog(KR): somaz.tistory.com Blog(EN): somaz.blog GitHub: github.com/somaz94

소개

안녕하세요, DevOps Engineer 이동형입니다.

안정적인 인프라 운영을 지향하며 모든 변경을 코드화(IaC)하고 장애 대응은 포스트모템으로 남겨 재발을 막아, 클라우드 비용을 20~50% 절감하고 운영 효율을 높여왔습니다. 단순 운영을 넘어 업무의 의미를 묻고 더 나은 방향을 찾는 습관이 가장 큰 강점입니다.

AWS·GCP 클라우드와 OpenStack 기반 온프레미스 PaaS에 걸친 인프라를 설계·구축합니다. Kubernetes로 컨테이너 운영을 표준화하고, Terraform·Ansible로 인프라를 코드화하며, GitLab CI·ArgoCD 중심의 자동 배포 파이프라인과 모니터링·로깅 스택을 운영합니다. 반복 업무는 Python·Bash로 자동화합니다.

데이터 파이프라인 자동화와 사내 GPU LLM 서비스로 데이터 엔지니어링·AI/ML 인프라 인접 영역도 직접 운영하고, git-bridge·Slack APK 봇 같은 사내 DevOps 도구도 직접 개발합니다.

클라우드·온프레미스를 아우르며 AWS→GCP 마이그레이션, AWS·온프레미스 하이브리드, 온프레미스 PaaS 구축을 단독 주도하며 AWS 비용 20%·GCP 50% 절감, 배포 시간 50~67% 단축, 수동 작업 90% 이상 자동화를 달성했습니다. 최근에는 신규 게임의 클라우드 프로덕션 런칭을 위한 AWS EKS 멀티클러스터를 구축하고 helmfile push 배포를 더 안정적인 자동 동기화 구조인 ArgoCD pull-GitOps로 전환했으며, 게임 DB 14초 멈춤 장애 해소와 Keycloak 중앙 SSO 통합으로 운영 효율을 한 단계 더 끌어올리고 있습니다.

앞으로도 인프라 아키텍처 고도화와 조직 생산성 향상에 기여하는 엔지니어가 되고자 합니다.

강점

본질을 묻는 사고 습관

- 업무의 본질을 묻는 습관 — '왜 이 작업이 필요한가'를 먼저 짚어 문제를 정의한 뒤 도구를 선택

체계적인 문서화와 지식 공유

- 모든 작업을 문서화·공유해 조직 지식 자산으로 축적

기록 기반의 지속적 개선

- 장애·인프라 변경·트러블슈팅을 포스트모템으로 기록해 팀이 함께 쓰는 자료로 축적 — 재발 방지·지속 개선

풀스택 인프라 경험

- 클라우드(AWS·GCP)·온프레미스·네트워크·CI/CD·모니터링을 아우르는 End-to-End 인프라

기술 스택

클라우드	AWS, GCP, OpenStack
컨테이너 오케스트레이션	Kubernetes, Docker, Cilium CNI, NGINX Gateway Fabric / Gateway API, Karpenter
Infrastructure as Code	Terraform, Ansible, Helm, Helmfile
CI/CD	GitHub Actions, GitLab CI/CD, ArgoCD, ArgoCD ApplicationSet, Jenkins, GitOps
모니터링 & 관측성	PLG Stack (Prometheus/Loki/Grafana), ELK Stack, EFK Stack, ECK Operator, Fluent Bit, Thanos, OpenTelemetry, Grafana Tempo
보안 & IAM	Keycloak (Operator, OIDC IdP), Vaultwarden, GitLab SSO (OIDC), External Secrets Operator, Sealed Secrets
자동화	Go, Python, Shell Scripting
데이터베이스	MySQL, PostgreSQL, Redis, MongoDB
OS & 서버	Ubuntu, Rocky Linux, CentOS, Debian, GPU Server

핵심 역량 (Core Competencies)

클라우드 인프라 설계 및 운영

AWS, GCP 기반 멀티·하이브리드 클라우드 아키텍처 설계 및 운영 경험

- AWS EKS·GCP GKE 프로덕션 운영·비용 최적화 (하이브리드 전환 20% 절감, GCP 마이그레이션 50% 절감, 절감분 신규 프로젝트 재투자)
- Kubernetes 스토리지 성능 최적화 (control-plane VM SSD 마이그레이션으로 etcd fsync·apiserver 응답 지연을 정상 범위로 회복, GitLab Runner 빌드 I/O 격리로 게임 DB COMMIT latency 14초 → ms 수준 회복)

CI/CD 파이프라인 구축 및 GitOps

GitOps 기반 배포 자동화로 개발 생산성 극대화

- 배포 시간 50~67% 단축, GitOps 지속 배포 전환으로 Time to Market(시장 대응 속도) 단축, 롤백 시간 95% 감소 (30분 → 1~2분)
- 계층형 공용 Toolchain Image + 자동 버전 사이클 구축으로 consumer repo 표준화, CI 첫 번째 잡 설치 시간 5분 → 약 10초(약 95% 단축), 사람 클릭 2회로 4단계 연쇄 전파 자동화

모니터링 및 옵저버빌리티 시스템 구축

장애 사전 감지 및 신속한 대응 체계 수립

- ELK → EFK Stack 전환 후 ECK Operator를 도입해 Elastic Stack 9.0을 CRD 선언형으로 재설계 (TLS 자동 회전, 롤링 업그레이드 자동화)
- Fluent Bit 로그 수집기 데이터 무결성·HA 강화 (SQLite offset DB + PVC, PodDisruptionBudget + preStop hook, 7개 Prometheus alert로 Pod 재시작 시 로그 누락/중복 0건 달성)

자동화 및 효율화

Python, Bash를 활용한 운영 자동화 및 개발 생산성 도구 구축

- Shell → Python(stdlib only) 마이그레이션 + 컴포넌트 단계별(wave) 순차 자동 배포·업그레이드 MR 자동 생성
- DP사 운영팀이 Kubernetes를 스스로 운영하도록 정착 — 6개월 교육 프로그램 설계·운영 (수강자 평균 10명·회당 4시간), 교육 자료 표준화로 타 고객사 교육에도 재활용

Kubernetes 플랫폼 운영 고도화 및 오픈소스 기여

네트워크·로깅 스택의 무중단 전환과 재사용 가능한 공용 차트 오픈소스 공개

- Ingress-nginx 다수 인스턴스 → NGINX Gateway Fabric 단일 컨트롤플레인 + 클래스별 Gateway CR 무중단 마이그레이션 (전체 cutover, wildcard TLS 통합)
- 오픈소스 Helm 차트(nginx-gateway-cr, elasticsearch-eck, kibana-eck) 및 composite GitHub Actions 다수 공개, ArtifactHub / GitHub Marketplace 등록

인프라 보안 & IAM

Keycloak Operator 기반 중앙 인증(SSO) 도입과 Vaultwarden 비밀번호 관리 시스템으로 사내 인증·시크릿 거버넌스 일원화

- Keycloak Operator + KeycloakRealmImport로 중앙 인증 시스템 도입 — 기존 GitLab 계정 그대로 로그인, ArgoCD / Harbor / Vaultwarden OIDC 통합(무중단, 검증 PASS), GitLab group → Keycloak mapper → ArgoCD role 단일 권한 흐름 확립
- Vaultwarden을 Kubernetes에 Helm 배포, GitLab SSO + 자동 백업(SQLite 덤프 스케줄링) + 대화형 restore 스크립트로 사용자 70명·시크릿 50건 중앙 관리

데이터 엔지니어링 및 사내 LLM 서비스

GCP 데이터 파이프라인부터 사내 LLM 서비스까지 직접 구축·운영

- BigQuery + Dataflow + Cloud Functions + Cloud Scheduler 조합으로 멀티 데이터 소스(MongoDB·CloudSQL·GA·Dune) → BigQuery → Google Sheets 자동 파이프라인 구축, 인프라 100% Terraform IaC
- 온프레미스 GPU 서버 위 Ollama + Open WebUI 기반 사내 LLM 서비스 Kubernetes 배포·운영

주요 경력 상세 (Professional Experience)

팬텀(콘크리트 스튜디오)

2024.10 ~ 현재

DevOps Engineer

게임 개발사의 인프라를 단독으로 책임지는 DevOps 엔지니어로, AWS 기반 클라우드와 온프레미스 서버를 설계·구축·운영하고 있습니다. dev·qa 서버는 온프레미스에, review·prod 서버는 AWS에 구축하여 하이브리드 아키텍처를 운영 중이며, 이 구조로 월 인프라 비용을 20%(\$60,000 → \$48,000) 절감하고 절감분을 신규 프로젝트에 재투자하는 선순환을 만들었습니다.
현재 서비스 중인 라이브 게임은 외부 퍼블리셔와 협업해 2차 기술지원 및 운영을 담당하고 있으며, 개발 중인 신규 게임 프로젝트의 개발 생산성 향상을 위해 온프레미스·AWS 인프라를 구축하고 있습니다.

프로젝트 1: AWS-온프레미스 하이브리드 클라우드 아키텍처 구축

기여도 100% (단독 설계 및 구현, 외부 퍼블리셔 운영팀 협업) 2024.10 ~ 현재 (운영 중)

문제

전체 인프라를 AWS에서 운영하던 중 월 \$60,000의 높은 클라우드 비용이 발생하여 비용 최적화가 필요했습니다.

접근 전략

워크로드별 비용을 분석한 결과, dev·qa 환경은 트래픽 변동이 적어 클라우드의 자동 스케일링이 불필요했고, review·prod 환경만 확장성과 안정성이 요구되는 상황이었습니다. 이에 dev·qa 환경은 온프레미스로 전환하고, review·prod 환경은 AWS에 유지하는 하이브리드 아키텍처를 설계했습니다. 양쪽 환경은 VPN 없이 역할을 분리해 독립 운영(앱 트래픽 분리)하며, 이미지 레지스트리도 환경별로 분리(dev·qa=Harbor, review·prod=ECR)해 아키텍처 차이에 맞춰 각 환경에서 독립적으로 빌드했습니다. 양쪽 환경 모두 Terraform과 Ansible을 활용한 IaC 통합으로 운영 표준을 일원화(신규 컴포넌트 IaC 100%, IAM 제외)하여, 운영 복잡도 증가 없이 일관된 인프라 관리 체계를 확립했습니다.

트리블슈팅

- EKS 환경에서 ALB Ingress 구성 시, 게임 클라이언트 버전별로 다른 백엔드로 라우팅해야 하는 요구사항 발생. ALB의 조건부 라우팅 규칙에서 커스텀 헤더 기반 라우팅과 Target Group을 조합하여 해결
- ArgoCD로 관리하는 프로덕션 서비스의 rolling deploy 중 ALB target deregistration delay(대상 등록 해제 대기)와 Pod SIGTERM 처리 시점이 어긋나 일부 세션 connection drop 발생.
3가지 조치로 connection drop 0건·zero-downtime 배포 달성 — (1) ALB drain 타이밍 정렬: preStop hook + readiness 우선 fail로 종료 중 신규 요청은 차단하되 in-flight 요청은 끝까지 처리, (2) RollingUpdate 전략을 replica 스케일에 맞춰 surge-first로 조정해 롤아웃 중 용량 유지, (3) 다중 replica 서비스에 PDB를 적용해 Karpenter consolidation(노드 통합·축소) 시 Pod 동시 축출 방지

성과

- 워크로드 분석 기반 인프라 최적화로 월 비용 20% 절감 (\$60,000 → \$48,000), 연간 약 \$144,000 절감분을 신규 프로젝트 인프라에 재투자하여 리소스 선순환 구조 구축
- 개발 서버 온프레미스 이전으로 월 약 \$1,000 AWS 리소스 비용 추가 절감 및 리소스 사용량 최적화
- 환경별 역할 분리로 운영 안정성 확보 및 장애 격리

프로젝트 2: 온프레미스 CI/CD 파이프라인 구축 및 고도화

기여도 100% (인프라 및 파이프라인 전체 담당) 2024.12 ~ 현재 (운영 중)

문제

개발자가 수동으로 빌드 후 서버에 배포하는 방식으로, 배포 시 평균 30분이 소요되었으며 휴먼 에러로 인한 장애가 빈번하게 발생했습니다.

접근 전략

GitLab CI와 ArgoCD를 조합한 GitOps 기반 자동 배포 환경을 구축했습니다. Kaniko로 Docker-in-Docker(privileged) 없이 Docker 이미지를 빌드하고, 멀티 환경(dev/qa/review/prod) 배포를 단일 파이프라인에서 관리하도록 구성했습니다.

트리블슈팅

- 환경 구분 없이 `selfHeal=true` 를 적용하면 prod 변경이 사람 승인 없이 자동 반영되는 리스크 → prod는 `selfHeal=false` + manual sync window로 배포를 게이트, dev/qa/review만 `selfHeal=true` 로 drift를 자동 보정, 모든 sync 이벤트를 Slack 알림으로 연동해 변경 이력을 끝까지 추적하면서 안정성과 긴급 대응 여지를 동시에 확보
- 데이터 업로드를 SSH scp/rsync 직결에서 S3 transit 버킷 스테이징 → SSM SendCommand 기반 `aws s3 sync --exact-timestamps` 로 전환해 SSH 키 제거 및 배포 계정 키 유출 리스크 차단

성과

- 배포 시간 67% 단축 (30분 → 10분), 데이터 배포 시 소스를 받아오는 단계를 `git fetch --depth=1 --filter=blob:none` 로 교체해 약 139초 → 약 30초로 단축
- GitOps 자동 배포를 환경별로 이원화 — dev/qa/review는 머지 기반 지속 배포(핵심 서비스 repo당 주 100회 내외)로 신규 피처의 Time to Market을 단축하고, prod는 수동 sync window 승인을 거쳐 주 5~7회만 릴리스해 안정성 확보
- 빌드·배포·설정 적용의 수동 단계를 파이프라인으로 대체해 휴먼 에러로 인한 배포 장애를 제거하고, cross-repo MasterData 파이프라인으로 데이터 변경 시 하위 서비스로 자동 전파(fan-out) + 순환 트리거(loop) 원천 차단

프로젝트 3: 중앙 로깅·관측 플랫폼 (로그 수집 → 무결성·HA → 제품 분석)

기여도 100% (로그 수집·무결성·제품 분석까지 전 과정 단독 설계·구축) 2024.11 ~ 현재

로그가 분산된 온프레미스 환경에서 시작해 중앙 집중식 로깅을 3단계로 고도화(ELK → EFK → ECK Operator)하고, 수집기 데이터 무결성·HA를 강화한 뒤, 축적된 EFK 로그 파이프라인을 게임 유저 리텐션 분석 플랫폼으로 확장했습니다. build → operate → improve → analytics로 이어지는 관측 플랫폼 전 과정을 단독으로 설계·운영했습니다.

3-1. 중앙 집중식 로깅 시스템 구축 (ELK → EFK → ECK Operator 3단계 고도화, 2024.11 ~ 현재)

온프레미스에 분산된 서버·컨테이너 로그로 장애 원인 파악이 느리고 통합 모니터링 체계가 없어, 로깅 스택을 **ELK → EFK → ECK Operator 3단계로 고도화**했습니다. 초기 ELK로 중앙 집중식 로깅을 구축한 뒤, Logstash를 노드마다 띄우기에는 JVM 메모리 부담이 커 수집은 경량 Fluent Bit DaemonSet에 맡기고 가공만 Fluentd로 분리한 EFK로 전환했습니다. 이후 Elastic 공식 Helm 차트가 2년 이상 정체되자 CRD 기반 선언형 관리(ECK Operator)로 옮기며 Elastic Stack을 8.5.1 → 9.0.0으로 major 업그레이드했고, 온프레미스 EFK 스택을 AWS(EKS)용 `--aws` 스택으로 템플릿화해 같은 구조를 이식하면서 로그 수명주기(ILM)와 S3 스냅샷 보관을 IRSA로 정적 키 없이 자동화했습니다.

성과: 로그 검색 시간 90% 단축·MTTR 개선, ECK 전환으로 TLS·계정·업그레이드 수동 부담 제거(CR spec.version 한 줄 롤링 업그레이드), 2년간 정체된 Elastic Stack의 major 업그레이드를 완료(8.5.1 → 9.0.0).

3-2. Fluent Bit 로그 수집기 데이터 무결성 & HA 강화 (2026.03 ~ 2026.05)

Fluent Bit Pod 재시작 시 tail offset 소실로 인한 로그 재색인·누락 위험을 SQLite offset DB·전용 PVC로 해결 — 재색인 0건·중복 0건, 7개 alert로 미감지 장애 차단

3-3. 게임 유저 리텐션 분석 플랫폼 (EFK 로그 → 제품 분석, 2026.06 ~ 현재)

장애·검색용 EFK 스택을 Elasticsearch Transform(5분)으로 이벤트를 사용자당 1행으로 집계하는 제품 분석 플랫폼으로 확장 — Kibana 12패널로 D+1~D+30 리텐션·코호트 제공, 별도 SaaS 없이 구현

프로젝트 4: 개발 생산성 도구 구축 (APK 배포 봇 · 문서 자동화 · LLM 서비스 · Git 미러링 · 정적 파일 서버)

기여도 100% 2025.02 ~ 현재

4-1. Slack APK 배포 자동화 봇 (4주, 2025.02)

Jenkins APK 빌드 완료 시 Slack으로 완료 메시지·다운로드 QR을 자동 전송하는 Python 봇을 Kubernetes에 배포 — QA 전달 90% 단축, 버전 혼동 제거

4-2. GitLab-Google Drive 문서 자동화 시스템 (2주, 2025.06)

GitLab Webhook·Google Drive API를 연동해 Push 시 마크다운을 Google Docs로 자동 변환·동기화하고 rclone `--checksum`으로 최적화 — 문서 관리 80% 단축(10분→2분), GitLab을 SSOT로 일원화

4-3. 사내 LLM 서비스 구축 (4주, 2025.11)

온프레미스 GPU 서버에 Ollama·Open WebUI를 Kubernetes로 배포해 사내 AI 모델을 직접 운영 — 개발팀 15명이 하루 100건 질의 처리

4-4. Git 저장소 미러링 도구 개발 + Retry API 후속 강화 (4주 초기 개발, 2026.02 ~ 2026.06)

CodeCommit·GitLab·GitHub 수동 동기화의 누락·지연을 없애려 Go로 양방향 Git 미러링 도구 git-bridge를 만들어 오픈소스 공개 — 저장소 10개·일 평균 30건 100% 자동화, webhook secret 노출 없는 Retry API 복구 수단과 `ref_overrides` 방향 고정으로 이력 되돌림 차단(테스트 전용 repo 6/6 PASS)

4-5. 정적 파일 서버 자체 개발 (오픈소스, 2026.03 ~ 현재)

halverneus/static-file-server가 디렉토리 UI·검색·ZIP 일괄 다운로드 없이 파일만 내려주고 업스트림 유지보수도 멈춰, Go로 정적 파일 서버를 직접 만들어 Apache-2.0으로 공개하고 자체 Helm chart·NFS로 배포해 완전 대체 — 일 평균 30건 다운로드·주 5회 APK 배포 처리

프로젝트 5: Kubernetes 클러스터 운영 고도화

기여도 100% (단독 설계 — 모니터링·업그레이드 자동화·Helm 관리 프레임워크·NGF 마이그레이션·OSS Helm 차트·스토리지 성능 최적화·Shell→Python 마이그레이션, 자동화 스크립트로 표준 절차를 코드로 남겨 → 팀 재사용 가능) 2025.12 ~ 현재

클러스터를 안정적으로 운영하기 위해 모니터링을 개편하고 K8s 업그레이드·Helm 관리를 자동화했으며, 네트워크 컨트롤러를 Gateway API 기반으로 마이그레이션했습니다.

NGF 마이그레이션과 ECK Operator 도입의 산출물을 OSS Helm 차트로 공개하고, control-plane / DB 노드의 SSD 마이그레이션 및 빌드 I/O 격리료 스토리지 성능 병목을 해소했으며, 인프라 배포/업그레이드 파이프라인을 단계별(wave) 순차 자동화 + Shell → Python 마이그레이션으로 정비했습니다.

5-1. 모니터링 & 알림 고도화 (2025.12 ~ 현재)

kube-prometheus-stack 기반 커스텀 알림·Grafana 대시보드를 설계하고 DB·node-exporter·Cilium 메트릭을 통합 — 9개 컴포넌트 그룹에 27개 커스텀 알림 운영, 심각도(severity)별로 알림을 나눠 보내고 상위 알림이 뜨면 하위 알림을 억제(inhibit)해 같은 장애의 중복 통보 제거

5-2. K8s 업그레이드 자동화 & Helm 차트 관리 프레임워크 (2025.12 ~ 현재)

Kubespray 업그레이드 자동화 스크립트와 다단계 헬스체크(노드·etcd·API Server)를 구축하고 Helm 프레임워크·표준 템플릿 동기화로 버전 관리를 표준화 — 검증 시간 90% 단축, 인증서 자동 갱신으로 만료 장애 제거

5-3. Ingress-nginx → NGINX Gateway Fabric(NGF) 마이그레이션 (2026.04)

온프레미스 ingress-nginx 다수 인스턴스를 NGF 2.x 단일 컨트롤플레인+클래스별 Gateway CR로 전환 — MetalLB IP 유지로 DNS·방화벽은 그대로 두고 장애 없이 cutover(클래스당 30~60초), wildcard 인증서로 수동 갱신 50% 감소

5-4. OSS Helm 차트 공개 — nginx-gateway-cr · elasticsearch-eck · kibana-eck (2026.04 ~ 2026.05)

NGF·ECK 산출물을 범용 차트(nginx-gateway-cr·elasticsearch-eck·kibana-eck)로 다듬어 Apache-2.0으로 ArtifactHub에 공개 — mgmt 클러스터를 cluster_uuid를 그대로 유지한 채 공용 OCI 차트로 무중단 전환

5-5. 스토리지 성능 최적화 — etcd / DB SSD 마이그레이션 + 빌드 I/O 격리 (2주, 2026.04 ~ 2026.05)

GitLab Runner buildx의 디스크 I/O 급증으로 HDD 위의 etcd·게임 DB가 초 단위 지연을 겪던 것을 진단해 control-plane·DB 워커를 SSD로 이전하고, 별도 물리 서버의 빌드 전용 VM에 buildx를 격리 — 게임 DB COMMIT 9~14초 → ms 수준 회복, 빌드 중 플레이 불가 장애 0건

5-6. 인프라 배포/업그레이드 자동화 + Shell → Python 마이그레이션 (2026.03 ~ 현재)

사내 Kubernetes 인프라 monorepo를 helmfile + shell로 직접 자동화해 운영해왔는데, 컴포넌트가 늘면서 shell + awk로는 환경별 분기와 복잡한 YAML 수정을 감당하기 어려워졌습니다. CI 파이프라인을 6 wave(bootstrap → core → security → db-redis → observability → apps)로 나누고 MR 변경분을 감지해 바뀐 컴포넌트만 helmfile로 배포하되 프로덕션 반영 전에는 사람이 승인하도록 했으며, 차트 신버전이 나오면 CI가 무엇이 바뀌었는지·설정값 차이·호환성이 깨지는 변경을 정리한 MR을 자동으로 열어주도록 업그레이드도 자동화했습니다. 기존 shell + awk 자동화는 Python(stdlib only)으로 옮기며 표준 템플릿·동기화/백업·CI 스크립트를 모듈로 나누고 단위 테스트를 붙였으며, CI에는 프로젝트 7의 helmfile-tools 이미지를 적용했습니다. 이 helmfile push 배포 모델은 이후 5-7의 ArgoCD pull-GitOps 컨트롤플레인으로 전환되었습니다(배포 모델 진화).

5-7. helmfile Push → ArgoCD Pull-GitOps 전환 (App-of-apps 컨트롤플레인, 2026.06 ~ 현재)

플랫폼 릴리스를 helmfile push로 배포하다 보니 배포 주체가 CI 러너에 묶여 있었고, 멀티클러스터 확장 시 클러스터별 드리프트를 선언적으로 바로잡을 pull 기반 GitOps 컨트롤플레인이 필요했습니다. Matrix + Git files 제너레이터 ApplicationSet이 각 컴포넌트의 `argocd/*.yaml` 마커를 읽어 Application을 자동 생성하도록 구성해 온프레미스와 AWS의 플랫폼 릴리스를 양쪽 클러스터에 등록했습니다.

무중단 adoption: 기존 helm 릴리스를 재생성 없이 그대로 ArgoCD 관리로 넘겨받아(adopt) 배포 순서를 보존했습니다.

안정화·최소권한: App-of-apps 연쇄 삭제 방지(prune-cascade) 안전장치로 대량 삭제를 차단하고, AppProject 최소 권한으로 장애 영향 범위(blast-radius)를 축소했습니다.

성과: 온프레미스·AWS 클러스터의 플랫폼 릴리스를 단일 컨트롤플레인에서 선언 관리, 기존 릴리스를 재생성 없이 넘겨받아 무중단 전환, prune-cascade·AppProject 최소 권한으로 blast-radius 축소.

프로젝트 6: 인프라 보안 체계 구축

기여도 100% 2026.04 ~ 현재

6-1. Vaultwarden 비밀번호 관리 시스템 (1주, 2026.04 ~ 현재)

Slack·이메일·개인 저장소에 흩어져 있던 사내 시크릿을 Vaultwarden으로 중앙화 — Kubernetes Helm 배포에 GitLab SSO(OIDC)·RBAC 연동, 사용자 70명·시크릿 50건 전환, SaaS 비용 0원

6-2. Keycloak Operator 기반 중앙 인증(SSO) 시스템 도입 (2026.04 ~ 현재)

ArgoCD / Harbor / Vaultwarden이 각각 GitLab을 OIDC IdP로 직접 사용하며 GitLab의 사용자·세션·MFA 정책에 의존하던 구조를, 로그인을 중개하는 중앙 broker(Keycloak)로 일원화했습니다. Keycloak Operator + KeycloakCR + KeycloakRealmImport를 PostgreSQL 백엔드와 함께 도입하고 GitLab을 Identity Provider로 brokering해, 사용자는 기존 계정 그대로 로그인합니다. Realm / Client / IdP / Group / Mapper는 부트스트랩 스크립트로 코드화했고, GitLab group을 Keycloak group mapper로 중개해 ArgoCD role에 연결해 권한 운영의 단일 기준을 Keycloak으로 통일했습니다.

성과: ArgoCD/Harbor/Vaultwarden OIDC 통합 완료(무중단, 검증 PASS), 도입 과정에서 발견된 5가지 함정(IdP providerId/scope/mapper config/Operator idempotency/Harbor user mapping)을 plan 문서에 정리해 재발 방지, 향후 LDAP 연동/MFA/세션 정책 중앙화의 기반 마련.

6-3. GitOps 시크릿 관리 — External Secrets Operator · Sealed Secrets (2026.05 ~ 현재)

DB 비밀번호를 Helm values 평문으로 Git에 커밋하던 구조를 External Secrets Operator로 AWS Secrets Manager와 연결해 ExternalSecret 참조로 전환, Harbor pull secret은 cluster-wide SealedSecret으로 표준화 — AWS SM을 시크릿의 단일 관리 기준으로 일원화

6-4. OpenVPN 원격 접속 서버 (2026.07)

iptime 공유기 내장 VPN의 동시 접속 5개 제한을 해소하기 위해 물리 서버에 OpenVPN 원격 접속 서버를 자체 구축 — easy-rsa PKI 기반 클라이언트별 EC 인증서(prime256v1) 인증 + tls-crypt 제어 채널 암호화·인증, AES-256-GCM-TLS 1.2+, full-tunnel 기본에 split-tunnel 전환을 지원하고, 터널 대역 10.8.0.0/24를 SNAT(MASQUERADE)로 사내 물리 서버망에 라우팅(동시 접속 최대 50 설정).

먹통 bash 설치 스크립트(PKI 빌드 → server.conf 렌더 → NAT systemd 유닛 → 서비스 기동)와 클라이언트 인증서 발급·폐기(revoke + CRL 갱신) 라이프사이클 스크립트로 운영을 자동화하고 관리자 Mac에서 SSH로 원격 제어

프로젝트 7: 공용 Toolchain Image 계층 구조 + 자동 버전 사이클

기여도 100% (아키텍처 설계 · CI 표준화 · 자동화 파이프라인 개발) 2025.09 ~ 2026.05

문제

GitLab CI 표준화 범위 안의 repo들에서 각 잡이 alpine 베이스에 helm·helmfile·kubect·crane·aws-cli 등을 매번 설치하면서 첫 번째 잡 준비 시간이 평균 5분에 달했고, 도구 버전이 consumer repo마다 흩어져 있어 신버전 추가 시 모든 consumer repo를 함께 손대야 했습니다. `:latest` 태그 의존으로 재현성도 떨어졌습니다.

접근 전략

Layer 1 (Mirror): 외부 레지스트리의 도구 이미지를 `crane copy` 로 사내 Harbor에 multi-arch manifest(여러 CPU 아키텍처를 묶은 이미지 목록)를 보존한 채 미러링.

Layer 2 (Custom): Layer 1 위에 도구가 사전 설치된 용량이 큰 이미지(helmfile-tools, node-build, go-build, rclone-backup, aws-cli-tools, kaniko-build, terraform-tools) 빌드.

Tier 1~2 SSOT(단일 관리 기준): `.toolchain_build_custom` 공용 template + `TOOLCHAIN_*_TAG` 중앙 변수 도입으로 consumer repo가 image tag를 직접 추적하지 않고 중앙 변수만 바라보도록 분리.

공용 CI 템플릿 SSOT: repo마다 복붙하던 CI 잡을 중앙 공용 CI 템플릿 repo로 추출해 각 repo가 `include` 로 참조하도록 전환. auth(키리스 AWS 인증) / build(kaniko) / deploy(ArgoCD) / backup(Google Drive) 잡 템플릿과 공용 앵커(git-ssh-rules·job-config·packages)를 단일 소스로 관리해 복붙 드리프트 제거.

Phase 4 자동 사이클: cron 트리거로 다음 4단계가 자동 진행되어 사람 click 2회로 연쇄 전파 완성:

- (1) upstream 신버전 감지
- (2) Layer 1 mirror auto-MR
- (3) Layer 2 rebuild + Tier 2 동기화 auto-MR
- (4) consumer repo 자동 전파

성과

- **CI 첫 번째 잡 준비 시간 5분 → 약 10초 (약 95% 단축)**, 사내 repo 전반에 도구 버전 일관성 100% 확보, 버전 거버넌스 4-tier(ARG → 중앙 변수 → consumer 분리 → lint drift 감지)로 흩어져 있던 버전 지정 지점을 한 흐름으로 정리
- Phase 4 cron 자동 사이클로 helm·helmfile·kubect·rclone 등 도구의 신버전 연쇄 전파를 사람 클릭 2회로 완료하고, minor-guard 정책(helm/helmfile/kubect)으로 호환성 깨짐 사전 차단
- 컴포넌트를 리스크 티어(T1 주간 / T2 격주 / T3 월간 + pre-flight)로 분류한 자동 업그레이드 파이프라인 구축(신버전 감지 → 자동 MR → Slack 알림 → wave 순차 적용, MAJOR_PIN 가드·T3 atomic 롤백), docker buildx로 amd64 + arm64 멀티아치 빌드 + 외부 이미지 crane Harbor 미러링으로 외부 레지스트리 의존 제거

프로젝트 8: AWS EKS 프로덕션 게임 런칭 플랫폼 구축 (신규 외부 퍼블리싱 게임)

기여도 100% (그린필드 EKS 단독 설계·구축) 2026.06 ~ 현재 (구축·운영 중)

문제

라이브 게임 환경(AWS review·prod)과 별개로, 신규 외부 퍼블리싱 게임은 온프레미스 개발 클러스터 하나로만 운영 중이었고, 프로덕션 런칭을 위해 확장성·가용성을 갖춘 별도 AWS EKS 프로덕션 환경이 필요했습니다. 온프레미스만으로는 글로벌 트래픽 대응과 오토스케일링에 한계가 있었습니다.

접근 전략

eu-central-1 리전에 그린필드 EKS 클러스터를 구축해 온프레미스 개발 + AWS 프로덕션의 멀티클러스터 하이브리드로 확장하고, 플랫폼 컴포넌트를 전부 GitOps(ArgoCD)로 선언 관리했습니다.

오토스케일링·네트워킹: Cluster Autoscaler 대신 Karpenter를 채택해(다수 스팟 family를 단일 프로비저너로 통합·빠른 노드 기동), 스팟 중단에 강한 다중 family 스팟 오토스케일링과 게임 서버 전용 on-demand NodePool을 구성하고, AWS Load Balancer Controller(ALB/NLB) + ExternalDNS(Route53) + wildcard ACM으로 인그레스·DNS·TLS를 자동화했습니다.

로깅·시크릿: eck-operator 기반 EFK 스택(Elasticsearch 3노드 3-AZ, EBS gp3, S3 스냅샷 SLM)을 HA로 구성하고, External Secrets Operator로 시크릿을 외부화했습니다. 모든 ServiceAccount 인증은 IRSA / Pod Identity로 처리해 정적 키를 제거했습니다.

인증·배포: ArgoCD에 GitLab OAuth SSO + 게임 스크립트 RBAC를 연동하고, 클라이언트 아티팩트는 Jenkins → S3 + CloudFront로, 서버 매니페스트는 S3 transit 버킷 + SSM SendCommand(SSH 없이 원격 명령 실행)로 bastion의 EFS 마운트에 전달했습니다. bastion을 Name 태그로 동적 탐색해 SSH 키 없이 배포했습니다.

트리블슈팅

- 게임 프로덕션 Pod가 실사용량의 약 4배에 달하는 메모리를 예약해 노드가 불필요하게 늘어남 → 실측 프로파일을 근거로 memory requests를 실제 사용량에 맞게 낮춰 노드 증설을 정상화하고 스팟 비용 효율 확보
- 롤링 배포 중 ALB가 아직 준비되지 않은 Pod로 트래픽을 전달하는 문제 → ALB Pod readiness gate + PodDisruptionBudget + graceful drain(연결을 안전하게 종료)을 조합해 무중단 배포 확보

성과

- 온프레미스 개발 클러스터 하나 → 온프레미스 + AWS 프로덕션 멀티클러스터 하이브리드로 확장, 신규 외부 퍼블리싱 게임의 클라우드 프로덕션 런칭 기반 확보
- 플랫폼 컴포넌트를 100% GitOps(ArgoCD)로 선언 관리하고, IRSA / Pod Identity + SSM SendCommand 키리스 배포로 정적 키 없는 인증 체계 확립 및 bastion SSH 키 유출 리스크 제거
- ArgoCD 앱 마커와 프로덕션 CI/CD 전환 문서로, 단독 구축이지만 팀이 동일 절차로 운영·재구축 가능하도록 인수인계 경로 확보

프로젝트 9: Observability 고도화 — 분산 추적 & Progressive Delivery

기여도 100% (단독 설계·구축 — OTel 추적 파이프라인·Argo Rollouts Canary·소프트 런칭 운영) 2026.06 ~ 현재

Project 8의 AWS EKS 프로덕션 게임 런칭 플랫폼 위에, 신규 외부 퍼블리싱 게임의 소프트 런칭에 대비해 관측과 배포를 보강했습니다.

OpenTelemetry 기반 엔드투엔드 분산 추적으로 로깅·메트릭만 있던 기존 관측 플랫폼에 '추적' 축을 채우고, Argo Rollouts로 무중단 Canary 배포를 도입했으며, 이 스택 위에서 소프트 런칭 실트래픽을 안정적으로 받아냈습니다.

9-1. OpenTelemetry 엔드투엔드 분산 추적 파이프라인 (OTel + Tempo)

AWS EKS에 OpenTelemetry Collector + Grafana Tempo(S3 저장, EKS Pod Identity로 정적 키 없음) + OpenTelemetry Operator(auto-instrumentation으로 앱 이미지 변경 없이 언어 SDK 주입)로 분산 추적 스택을 구축했습니다. 여기서 핵심은 샘플링 지점을 나눈 것입니다.

RED 메트릭은 샘플링 이전 100% span에서 생성해 정확도를 확보하고, trace 저장 단계에만 tail sampling(모든 에러 + 500ms 초과 + 나머지 10%)을 적용해 저장 비용을 절감했습니다. Grafana에서 Metrics ↔ Traces ↔ Logs 3-signal을 서로 오갈 수 있게 연결해, 로깅·메트릭만 있던 관측 플랫폼에 비어 있던 '추적' 축을 채웠습니다.

9-2. Argo Rollouts 기반 Progressive Delivery (Canary)

기존 Blue-Green은 승격 시 ALB target group을 한 번에 교체해, 순간 healthy target이 0이 되고 그 틈에 ALB가 503을 반환하는 경향이 있었습니다 (2026-07-14 관측). 그래서 Argo Rollouts Canary로 전환했습니다.

같은 ALB target group에서 `maxUnavailable: 0` + add-before-remove(새 Pod를 먼저 등록한 뒤 구 Pod 제거)로 old/new가 순간 공존하며 넘어가 503 레이스를 제거했고, 게임 서비스가 stateless HTTP(JWT + Redis/TypeORM로 상태 외부화)라 전환 중 old/new에 트래픽이 섞여 들어와도 안전합니다. 공용 base-aws Helm 차트에서 배포 전략을 opt-in으로 켜고 끄도록 템플릿화해, 프로덕션 게임 메인 서비스 1개만 전환하고(형제 내부 서비스는 일반 Deployment 유지), 컨트롤러는 leader-election HA + PDB로 구성해 노드 drain·Karpenter consolidation 중에도 돌아움이 멈추지 않습니다.

9-3. 소프트 런칭 운영 (2026.07 ~ 현재)

2026.07.14(KST) 소프트 런칭을 개시하고, 앞의 스택 위에서 신규 외부 퍼블리싱 게임을 제한 공개로 열어 그 소규모 실트래픽을 무중단 Canary 배포와 3-signal 관측(실 로그에서 trace_id/span_id 상관관계 검증)으로 안정적으로 받아냈습니다.

HPA·Karpenter 오토스케일링은 2026 Q4 글로벌 정식 런칭을 대비해 프로비저닝을 완료했습니다(소프트 런칭 트래픽에서는 스케일업 미발생).

너디스타

2023.03 ~ 2024.07

DevOps Engineer

게임 및 블록체인 기반 스타트업에서 DevOps 엔지니어로 근무하며, AWS에서 GCP로 대규모 클라우드 마이그레이션 프로젝트를 총괄했습니다.

프로젝트 1: AWS → GCP 대규모 클라우드 마이그레이션 및 CI/CD 구축

기여도: 인프라 설계·마이그레이션 총괄 70% (나머지 30% 는 서버 개발팀 application 마이그레이션 협업), CI/CD 파이프라인 100% 10개월 (2023.03 ~ 2023.12)

문제

AWS 비용 지속 상승(월 \$10,000+, NAT Gateway·데이터 전송 비용 높음), GCP Startup Program 참여 기회, 멀티 리전 게임 서비스를 위한 GCP 글로벌 LB 확보

접근 전략

Shared VPC (Host / Service Project 분리)와 Workload Identity Federation을 도입해 GitHub Actions가 서비스 계정 키 없이 임시 토큰으로 GCP 리소스에 인증하도록 구성. DNS는 서브도메인 위임 (Hosting.kr → AWS Route53 → GCP Cloud DNS)으로 사전 검증 후, 최종 NS 변경 시점에만 점검 공지 후 cutover.

Filestore의 SSD 최소 2.5TB 비용 문제는 Compute Engine + pd-balanced 1TB 디스크 기반 NFS 서버 자체 구축으로 우회. Cloud Armor로 region 차단 + IP 화이트리스트 WAF(웹 방화벽) 정책 구성.

트러블슈팅

- AWS ALB → GCP 글로벌 LB 전환 시 AWS ALB는 헬스체크에 실패한 백엔드에도 트래픽을 통과시키지만 GCP LB는 차단하는 차이로 라우팅 이슈. GCP NEG 구조 분석 후 헬스체크·백엔드 재구성
- GKE Ingress + BackendConfig의 healthcheck requestPath 응답 누락으로 503 발생. 해당 경로 응답 보장 + ManagedCertificate / FrontendConfig 정렬로 해결
- CDN 마이그레이션 후 HTTP → HTTPS 리다이렉트 누락으로 게임 클라이언트 파일 다운로드 실패. URL Map을 HTTP / HTTPS 두 개로 분리 (`default_url_redirect.https_redirect=true`) 후 해결
- DNS를 Cloud DNS로 위임 후 Google Search Console 도메인 미등록 문제 발생. Search Console 발급 verification TXT 레코드를 Cloud DNS에 추가해 소유권 검증 후 재등록으로 해결

성과

- 클라우드 비용 50% 절감 (월 \$10,000 → \$5,000), 배포 시간 50% 단축, 롤백 95% 감소
- 전체 GCP 인프라 Terraform IaC화 (TFLint/Checkov 정적 분석, Terratest 단위 테스트 도입), 주간 배포 횟수 3배 증가, Git 커밋 기반 배포 이력 100% 추적
- 리소스 복사 방식으로 다운타임 최소화, 최종 DNS 전환은 게임 서비스 특성을 반영해 2시간 내외 점검 공지 후 전환
- Workload Identity Federation 도입으로 GitHub Actions의 GCP 서비스 계정 키 관리 부담 제거 및 키 유출 리스크 원천 차단

프로젝트 2: 게임 데이터 수집 자동화 시스템 개발

기여도 100% 3개월 (2024.01 ~ 2024.03)

문제

게임·블록체인 데이터를 수동으로 수집하여 분석가가 매일 2~3시간 소요, 휴먼 에러로 인한 데이터 누락 빈번 발생

접근 전략

MongoDB · CloudSQL · Google Analytics · Dune 등 멀티 데이터 소스를 BigQuery로 적재 (MongoDB는 Dataflow ETL, CloudSQL은 직접 connection, GA/Dune은 API) 후, Cloud Functions + Cloud Scheduler가 Daily / Monthly 단위로 자동 트리거하고 Google Sheets API로 분석가용 시트에 자동 적재. BigQuery 단의 중복 데이터는 별도 Cloud Function으로 정기 제거.

전체 인프라 (Dataset · Cloud Function · Scheduler · Storage · Artifact Registry)는 Terraform IaC로 관리.

성과

- 데이터 수집 자동화 95% 달성, 휴먼 에러 제거로 데이터 누락률 0%

프로젝트 3: PLG Stack 기반 모니터링 시스템 구축

기여도 100% 4개월 (2024.04 ~ 2024.07)

문제

Kubernetes 클러스터·애플리케이션 실시간 모니터링 체계가 없어 사후 대응만 가능했고, 장애 원인 파악에 과도한 시간이 소요됨

성과

- SLI/SLO 수립과 Alertmanager 알림 노이즈 정리, 장애 사전 감지 체계 확립, 장애 대응 시간 단축, 서비스 가용성 향상

아이오차드

2022.04 ~ 2023.03

Infra Engineer

PaaS(Kubernetes + OpenStack + Ceph) 기반 인프라 솔루션을 고객사에 구축하고 기술 지원을 제공했고, 금융권 및 공공기관을 대상으로 온프레미스 클라우드 인프라를 설계·구축하고 운영 교육도 직접 맡았습니다.

주요 프로젝트: PaaS 솔루션 구축 · DP사 교육 · 인증서 자동화 · Ansible 구성 관리

성과

- 5개 고객사에 PaaS 솔루션 구축, Kubernetes HA 기반 안정적 운영
- DP사 운영팀 Kubernetes 자체 운영 체계 확립 (6개월, 수강자 평균 10명 · 회당 4시간)
- 인증서 갱신 주기 10배 연장 (1년 → 10년), 연간 약 50시간 운영 부담 절감 (고객사 10곳 기준)
- Ansible 기반 서버 프로비저닝 및 구성 관리 자동화로 환경 불일치(Configuration Drift) 0건 달성

이테크시스템

2022.01 ~ 2022.04

System Engineer

하드웨어 서버 및 SAN 스토리지 구축. DevOps 분야로 커리어 전환을 위해 이직.

오픈소스 프로젝트

Kubernetes CRD

[k8s-namespace-sync](#) — Kubernetes 네임스페이스 리소스를 클러스터 간 자동 동기화

[helios-lb](#) — Kubernetes용 커스텀 로드밸런서 컨트롤러

[network-policy-generator](#) — Kubernetes NetworkPolicy 자동 생성 컨트롤러

Helm Charts

[nginx-gateway-cr](#) — NGINX Gateway Fabric용 테넌트 리소스 Helm 차트

[elasticsearch-eck](#) — ECK Operator용 Elasticsearch CR Helm 차트

[kibana-eck](#) — ECK Operator용 Kibana CR Helm 차트

[certmanager-letsencrypt](#) — cert-manager용 Let's Encrypt 리소스 Helm 차트

GitHub Actions

[Compress Decompress Action](#) — 워크플로우 내 파일 압축/해제

[Image Tag Updater](#) — YAML/Helm 파일 이미지 태그 자동 업데이트

[Go Git Commit Action](#) — Go 기반 Git 커밋 생성 및 푸시

[Extract Commit Action](#) — Git 커밋 SHA, 메시지, 작성자 정보 추출

[Multi Git Mirror Action](#) — 여러 Git 저장소 간 미러링 자동화

[Kube Diff Action](#) — Kubernetes 리소스 변경사항 비교 및 리포트

[Major Tag Action](#) — 메이저 버전 태그(v1)를 최신 semver 릴리즈로 자동 갱신

[Go Changelog Generator](#) — Conventional Commits 기반 체인지로그 자동 생성

[Environment Output Setter](#) — 여러 환경 변수·워크플로우 출력을 변환과 함께 설정

[Ternary Operator Action](#) — 최대 10개 조건 평가로 동적 출력 설정

[Contributors Action](#) — 저장소 데이터로 기여자 목록(HTML·MD·이미지) 자동 생성

[Kube Events Action](#) — Kubernetes 클러스터 이벤트 점검·필터링·PR 코멘트

[Helm Kustomize Lint Action](#) — Helm 차트 lint·검증(yamllint·helm lint·template·kubeconform)

[Helm OCI Push](#) — Helm 차트를 OCI 레지스트리(GHCR·ECR·Harbor·Quay)로 푸시

Ansible Galaxy

[Ansible K8s IAC Tool](#) — K8s 및 IaC 도구 자동 설치 컬렉션

[Ansible User Management](#) — Linux 사용자 및 SSH 키 관리 역할

DevOps Tools

[bash-pilot](#) — AI 기반 Bash 명령어 어시스턴트 CLI 도구

[git-bridge](#) — Git 저장소 간 미러링 및 동기화 CLI 도구

[static-file-server](#) — 디렉토리 UI·파일 프리뷰·검색·ZIP 일괄 다운로드 지원 Go 기반 정적 파일 서버

[kube-diff](#) — 매니페스트(YAML·Helm·Kustomize)를 라이브 클러스터와 비교하는 CLI

[kube-events](#) — Kubernetes 이벤트 조회·요약 CLI (그룹핑·필터·컬러)

Chrome Extensions

[Dev Toolkit](#) — 개발자 유틸리티 모음

[QRify](#) — URL/텍스트 QR 코드 즉시 생성

학력

충남대학교 행정학부 (학사편입)	2018.03 ~ 2020.08
국가평생교육진흥원 경영학사 (학점은행제)	2016.09 ~ 2018.02

자격증

정보처리기사	2021.11
AWS Certified Solution Architect - Associate (만료)	2021.11
네트워크 관리사 2급	2021.10
리눅스 마스터 2급	2021.10
컴퓨터활용능력 1급	2021.04
TOEIC Speaking Test - 130점/Intermediate Mid 3 (만료)	2021.02

스터디

기술 블로그 운영 (somaz.blog, 영어) — DevOps · Kubernetes · IaC 글로벌 독자 대상 정기 작성	2025.01 ~ 현재
기술 블로그 운영 (somaz.tistory.com, 한국어) — DevOps · Kubernetes · IaC 학습/운영 노트 꾸준한 작성	2023.04 ~ 현재
T101(Terraform) 스터디	2023.08 ~ 2023.10
AEWS(EKS) 스터디	2023.04 ~ 2023.06
PKOS(Kubernetes) 스터디	2023.01 ~ 2023.02
Cloud Security 교육	2022.02 ~ 2022.04
보안솔루션 운영 전문가 양성과정 (국비교육)	2021.07 ~ 2021.12